Werkzeuge · Warm-up-Karten

# Warm-up-Karten Welt 6 – Der Endgegner

„Endgegner-Mix“ für die Generalprobe und die Klausur (DS 37–40): Jede Karte W6-01 bis W6-14 greift einen Abi-Klassiker aus den Welten 0 bis 5 auf – Methodenkopf, Objektkarten, binäre Suche, Sortieren, Rekursionstabelle, Stapel und Schlange, 2D-Reihung, Binärbaum – und baut gezielt eine der **Top 10 Punktverluste** P1 bis P10 als Falle ein. Die P-Nummer steht in der ersten Zeile jeder Karte. Alle Daten sind neu (Thronsaal des Endgegners); Probeklausur, AB1 und Klausuren verwenden andere Kontexte.

### So setzt du die Karten ein

- **DS 38 – Fehlerjagd plus:** Nach Teil 4 des AB1 (Korrektur und Fehlerjagd) ziehen Teams, die ihre Top 3 erledigt haben, die Karte zu ihrer häufigsten P-Nummer (Tabelle unten).
- **Warm-up-Nachtest (vor der Klausur, ca. 20 min):** Jede Person löst ohne Gerät die drei Karten zu den Top-3-P-Nummern ihres Teams (W6-01 bis W6-10, je eine Karte pro P). Jede Falle, in die man trotzdem tappt, gibt einen Strich in der Spalte „Warm-up-Nachtest Σ“ der Team-Strichliste (AB1, Teil 3). Der Vergleich mit der Strichliste aus DS 38 zeigt, welche Fallen kleiner geworden sind.
- **Klausur-Training:** W6-11 bis W6-14 mischen mehrere Fallen (Hausaufgabe zwischen DS 38 und der Klausur, Kontrolle mit dem Lösungsteil).
- Digital im **Warm-up-Trainer von Algo Quest Lab** (Welt-Filter „Welt 6“; Leitner-Prinzip, ohne Punkte) – ideal für den Wiederholungsplan bis zum Abitur. Passend dazu im Lab: Level 6.1 „Fallenjagd I – P1 bis P5“ und 6.2 „Fallenjagd II – P6 bis P10“.

## Kartenübersicht

| Nr. | Falle | Kategorie | Titel | Klassiker | Min. | Einsatz |
|---|---|---|---|---|---|---|
| W6-01 | P1 | Fehler im Java-Code | Die Welle bricht | Welt 4 | 6 | Nachtest P1 |
| W6-02 | P2 | Methode implementieren | Die Welle bleibt stehen | Welt 4 | 8 | Nachtest P2 |
| W6-03 | P3 | Methodenkopf und Signatur | Falsch beschriftete Zauberformeln | Welt 0 | 6 | Nachtest P3 |
| W6-04 | P4 | Fehler im Java-Code | Der Rechenfehler des Orakels | Welt 0/4 | 6 | Nachtest P4 |
| W6-05 | P5 + P1 | Fehler im Java-Code | Das Siegel der Ritter | Welt 0/2 | 7 | Nachtest P5 |
| W6-06 | P6 + P8 | Tracetabelle | Die Schatzsuche endet zu früh | Welt 2 | 7 | Nachtest P6 |
| W6-07 | P7 | Tracetabelle | Das Muster des Drachen | Welt 3 | 8 | Nachtest P7 |
| W6-08 | P8 | Tracetabelle | Das Protokoll des Kerkermeisters | Welt 4 | 8 | Nachtest P8 |
| W6-09 | P9 + P6 | Fehler im Struktogramm | Der Bauplan mit Java-Resten | Welt 0/4 | 7 | Nachtest P9 |
| W6-10 | P10 | Funktionsweise erläutern | Behaupten ist nicht begründen | Welt 5/2 | 8 | Nachtest P10 |
| W6-11 | P10 | Objektkarten und Referenzen | Zwei Namen, ein Held | Welt 1 | 8 | Klausur-Training |
| W6-12 | P6 + P10 | Funktionsweise erläutern | Das Sortier-Duell der Teams | Welt 2 | 8 | Klausur-Training |
| W6-13 | P7 + P1 | Methode implementieren | Die Einzelgänger im Truhenbaum | Welt 5 | 8 | Klausur-Training |
| W6-14 | P1 + P6 | Verändern und Erweitern | Fallen im Thronsaal | Welt 3 | 8 | Klausur-Training |

## Welche Karte trainiert welchen Punktverlust?

| P | Punktverlust (Kurztitel) | Nachtest-Karte | weitere Karten |
|---|---|---|---|
| P1 | Laufzeitfehler nicht ausgeschlossen | W6-01 | W6-05, W6-13, W6-14 |
| P2 | Struktur zerstört statt unverändert | W6-02 | – |
| P3 | Signatur verfehlt | W6-03 | – |
| P4 | Typen und Casts | W6-04 | – |
| P5 | Zeichenketten mit == statt equals | W6-05 | – |
| P6 | Off-by-one | W6-06 | W6-09, W6-12, W6-14 |
| P7 | Rekursion | W6-07 | W6-13 |
| P8 | Trace-/Dokumentationstabelle | W6-08 | W6-06 |
| P9 | Struktogramm nicht nach EH §5 | W6-09 | – |
| P10 | Operator verfehlt | W6-10 | W6-11, W6-12 |

Druckhinweis: Kartenseiten einseitig drucken, an den gestrichelten Linien schneiden (2 Karten im Format A5 je Seite), ggf. laminieren. Der Lösungsteil ist für die Kontrollstation bzw. das Lehrerexemplar gedacht. Alle Werte, Tabellen und Compiler-Aussagen wurden per Programm geprüft (javac 21 mit Nachbau der Bibliothek „Niedersachsen“).

WARM-UP W6-01 Fehler im Java-Code P1

Welt 6 – Endgegner · ⏱ 6 min

## Die Welle bricht

Falle P1 – Laufzeitfehler nicht ausgeschlossen. Klassiker aus Welt 4 (Stapel und dynamische Reihung nach EH §4, Abi 24H-B e, 25N-B l). Beide Methoden gehören zur Klasse Thronsaal; Monster ist die Klasse aus Welt 1 (getName(), getLeben(), getStaerke()). Bibliothek „Niedersachsen“: Inhalte werden gecastet.

**a)** Der Stapel treffer enthält (oben zuerst) 12, 7, 30. Gib die Rückgabe von schadenOben(treffer, 2) an und gib einen Aufruf von schadenOben mit diesem Stapel an, der zu einem Laufzeitfehler führt.

**b)** Die dynamische Reihung welle enthält Monster mit den Lebenspunkten 25, 40, 90 (Index 0 bis 2). Gib die Rückgabe von ersterUeber(welle, 30) an und einen Aufruf mit dieser Reihung, der zu einem Laufzeitfehler führt. Benenne jeweils die Ursache.

**c)** Korrigiere beide Methoden so, dass sie die Vorgaben in den Kommentaren erfüllen und kein Laufzeitfehler mehr möglich ist.

```
// Summe der obersten n Treffer (Stapel vom Inhaltstyp Ganzzahl);
// liegen weniger als n Treffer im Stapel, werden alle addiert.
// Der Stapel darf dabei geleert werden.
public static int schadenOben(Stack treffer, int n) {
    int summe = 0;
    for (int i = 0; i < n; i++) {
        summe = summe + (Integer) treffer.pop();
    }
    return summe;
}

// Index des ersten Monsters mit mehr als grenze Lebenspunkten;
// -1, wenn es keines gibt
public static int ersterUeber(DynArray welle, int grenze) {
    int i = 0;
    while (((Monster) welle.getItem(i)).getLeben() <= grenze) {
        i++;
    }
    return i;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-01

WARM-UP W6-02 Methode implementieren P2

Welt 6 – Endgegner · ⏱ 8 min

## Die Welle bleibt stehen

Falle P2 – Struktur zerstört statt unverändert. Klassiker aus Welt 4 (Muster „unverändert lassen“ mit Hilfsschlange, Boss 4 c, Abi 25H-B b, 26H-B e). Die Gegnerwelle des Endgegners ist eine Schlange vom Inhaltstyp Monster (Klasse aus Welt 1).

**a)** Implementiere staerkstes(q: Schlange vom Inhaltstyp Monster): Monster. Die Methode liefert das Monster mit der größten Stärke, bei Gleichstand das vordere, bei leerer Schlange null. Nach dem Aufruf muss q unverändert sein (dieselben Monster in derselben Reihenfolge).

**b)** Gib die Rückgabe für die Welle q (vorn zuerst) Ghul (Leben 40, Stärke 9), Wyvern (80, 14), Mimik (25, 14), Golem (120, 11) an.

**c)** Ein Mitschüler sammelt die Monster beim Durchlauf in einem Hilfsstapel und schichtet am Ende alles vom Stapel zurück in q. Gib die Reihenfolge von q nach seinem Aufruf an und erläutere, warum das gegen die Vorgabe verstößt.

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-02

WARM-UP W6-03 Methodenkopf und Signatur P3

Welt 6 – Endgegner · ⏱ 6 min

## Falsch beschriftete Zauberformeln

Falle P3 – Signatur verfehlt. Klassiker aus Welt 0 (Signatur in EH-Notation ↔ Java-Methodenkopf, Boss 0 a) – im Abitur beginnt jede Implementierung damit. Alle Operationen sind Klassenoperationen (static); Held, Monster und Gegenstand sind die Klassen aus Welt 1.

**a)** Unter jeder Vorgabe (1) bis (4) steht der Methodenkopf, den jemand dazu geschrieben hat. Benenne in jedem Kopf die Abweichungen von der Vorgabe und gib den richtigen Kopf an.

**b)** Die Methode istBesiegbar (unten) hat jetzt den richtigen Kopf, wird aber vom Compiler trotzdem nicht akzeptiert. Erläutere den Grund und korrigiere die Methode so, dass sie false liefert, wenn keine der beiden Bedingungen erfüllt ist.

**c)** Gib einen Aufruf von istBesiegbar an, der genau über die zweite Bedingung true liefert.

```
(1) beuteWert(schatz: Reihung von Gegenstand): Ganzzahl
    public static int beuteWert(Gegenstand schatz)
(2) verteileGold(gold: Ganzzahl, anzahl: Ganzzahl): Kommazahl
    public static int verteileGold(int gold, int anzahl)
(3) gegnerListe(welle: Schlange vom Inhaltstyp Monster):
        dynamische Reihung vom Inhaltstyp Monster
    public static Queue gegnerListe(DynArray welle)
(4) istBesiegbar(boss: Monster, held: Held): Wahrheitswert
    public void istBesiegbar(Monster boss, Held held)

public static boolean istBesiegbar(Monster boss, Held held) {
    if (held.angriffswert() >= boss.getLeben()) {
        return true;
    } else if (held.getLeben() > 2 * boss.getStaerke()) {
        return true;
    }
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-03

WARM-UP W6-04 Fehler im Java-Code P4

Welt 6 – Endgegner · ⏱ 6 min

## Der Rechenfehler des Orakels

Falle P4 – Typen und Casts. Klassiker aus Welt 0 (Werkstatt: Datentypen, ganzzahlige Division, Zeichen) und Welt 4 (Casten bei den Klassen der Bibliothek „Niedersachsen“). Gegeben sind karte, eine Reihung von Reihung von Zeichen mit karte[2][3] = '#', und schaden, eine dynamische Reihung vom Inhaltstyp Ganzzahl mit den Werten 12, 7, 30. Der ASCII-Wert von '#' ist 35.

**a)** Entscheide für jede der Zeilen 1 bis 9, ob der Compiler sie akzeptiert. Korrigiere die fehlerhaften Zeilen.

**b)** Gib für die akzeptierten (bzw. korrigierten) Zeilen den Wert der Variablen an.

**c)** Ändere Zeile 2 durch einen einzigen Cast so, dass quote die Trefferquote in Prozent enthält.

```
1  int treffer = 7; int versuche = 8;
2  double quote = treffer / versuche * 100;
3  double quote2 = 100.0 * treffer / versuche;
4  int mittel = (12 + 7 + 30) / 3;
5  char c = karte[2][3];
6  boolean wand = c == "#";
7  int erster = schaden.getItem(0);
8  String s = "Feld " + c + 1;
9  int code = c + 1;
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-04

WARM-UP W6-05 Fehler im Java-Code P5 P1 Welt 6 – Endgegner · ⏱ 7 min

## Das Siegel der Ritter

Falle P5 – Zeichenketten mit == statt equals + P1 – Laufzeitfehler nicht ausgeschlossen. Klassiker aus Welt 0 (Vorratskammer: beginntMit) und Welt 2 (Suche mit equals). anzahlMitTitel(namen: Reihung von Zeichenkette, titel: Zeichenkette): Ganzzahl liefert, wie viele Namen mit dem Titel beginnen (Groß- und Kleinschreibung zählt); istGast(gaeste: Reihung von Zeichenkette, name: Zeichenkette): Wahrheitswert prüft, ob der Name in der Gästeliste des Thronsaals steht.

**a)** Nenne die Stellen, die gegen die erlaubten Zeichenketten-Operationen der Ergänzenden Hinweise verstoßen oder Zeichenketten fachlich falsch vergleichen.

**b)** Erläutere, warum istGast in der Online-IDE das erwartete Ergebnis liefert, im Abitur aber als falsch bewertet wird.

**c)** Schreibe beide Methoden neu – nur mit length(), charAt(), substring() und equals(). Gib anzahlMitTitel für namen = {"Sir Aldo", "Sira", "Si", "sir Bert", "Sir Cato"} und titel = "Sir " an. Welcher Name löst einen Laufzeitfehler aus, wenn du substring ohne Längenprüfung verwendest (P1)?

```
public static int anzahlMitTitel(String[] namen, String titel) {
    int anzahl = 0;
    for (int i = 0; i < namen.length; i++) {
        if (namen[i].startsWith(titel)) {
            anzahl++;
        }
    }
    return anzahl;
}

public static boolean istGast(String[] gaeste, String name) {
    for (String g : gaeste) {
        if (g == name) {
            return true;
        }
    }
    return false;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-05

WARM-UP W6-06 Tracetabelle P6 P8 Welt 6 – Endgegner · ⏱ 7 min

## Die Schatzsuche endet zu früh

Falle P6 – Off-by-one + P8 – Trace-/Dokumentationstabelle. Klassiker aus Welt 2 (binäre Suche iterativ mit lo, hi, mid; Abi 22N-B d, 24N-A d/e). Die Nummern der Schatztruhen liegen aufsteigend sortiert in r = {4, 9, 15, 22, 31, 38, 47, 56, 63} (Index 0 bis 8).

**a)** Dokumentiere den Aufruf suche(r, 63) in einer Tracetabelle mit den Spalten Durchlauf, lo, hi, mid, r[mid] (eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle: vor der Schleife und am Ende jedes Durchlaufs) und gib die Rückgabe an.

**b)** Benenne den Off-by-one-Fehler und korrigiere ihn. Gib alle Werte aus r an, die von der fehlerhaften Methode nicht gefunden werden.

**c)** Gib für die korrigierte Methode die maximale Anzahl an Schleifendurchläufen bei 9 Werten und bei 1000 Werten an.

```
public static int suche(int[] r, int x) {
    int lo = 0;
    int hi = r.length - 1;   // → Tabellenzeile (Startwerte)
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (r[mid] == x) {
            return mid;
        }
        if (r[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }   // → Tabellenzeile
    }
    return -1;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-06

WARM-UP W6-07 Tracetabelle P7

Welt 6 – Endgegner · ⏱ 8 min

## Das Muster des Drachen

Falle P7 – Rekursion. Klassiker aus Welt 3 (Rekursionstabelle mit Spalte Rekursionstiefe, Erstaufruf = Tiefe 0; Abi 24N-B h, 25H-B i, 26H-B g).

**a)** Dokumentiere den Aufruf muster(6) in einer Rekursionstabelle mit den Spalten Rekursionstiefe, Aufruf und Rückgabewert und gib das Ergebnis an.

**b)** Jemand ersetzt den Basisfall (Zeilen 2 bis 4) durch if (n == 1) { return "*"; }. Gib an, für welche Aufrufe muster(n) mit n ≥ 0 sich dadurch etwas ändert, und erläutere, was dann passiert.

**c)** Jemand ersetzt Zeile 6 durch die zwei Anweisungen muster(n - 1); return "*";. Gib muster(6) für diese Variante an und benenne den Fehler.

**d)** Begründe, welche Rekursionstiefe der letzte Aufruf bei muster(16) erreicht.

```
1  public static String muster(int n) {
2      if (n == 0) {
3          return "";
4      }
5      if (n % 2 == 1) {
6          return muster(n - 1) + "*";
7      }
8      return "-" + muster(n / 2);
9  }
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-07

WARM-UP W6-08 Tracetabelle P8

Welt 6 – Endgegner · ⏱ 8 min

## Das Protokoll des Kerkermeisters

Falle P8 – Trace-/Dokumentationstabelle. Klassiker aus Welt 4 (Mischaufgabe mit Schlange und Stapel, Abi 22N-B a, 24H-B e). Die Schlange q enthält zu Beginn (vorn zuerst) 5, 8, 3, 6, der Stapel s ist leer. Vorgabe für die Tracetabelle: nur Variablen; eine Zeile mit den Anfangswerten vor der Schleife, dann eine Zeile an jeder mit „// → Tabellenzeile“ markierten Stelle; alle Spalten in jeder Zeile ausfüllen.

**a)** Unter dem Code steht die Tabelle einer Mitschülerin. Nenne drei Verstöße gegen die Vorgabe.

**b)** Dokumentiere den Ablauf korrekt mit den Spalten Durchlauf, x, q (vorn zuerst), s (oben zuerst), summe und gib den Wert von summe nach der Schleife an.

```
while (!q.isEmpty()) {
    int x = (Integer) q.dequeue();
    if (x % 2 == 0) {
        s.push(x);
    } else {
        summe = summe + x;
        if (!s.isEmpty()) {
            q.enqueue((Integer) s.pop() + 1);
        }
    }   // → Tabellenzeile
}

Tabelle der Mitschülerin:
x | q       | s | summe
5 | 8, 3, 6 |   | 5
3 | 6, 9    |   | 8
9 | 7       |   | 17
7 |         |   | 24
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-08

WARM-UP W6-09 Fehler im Struktogramm P9 P6

Welt 6 – Endgegner · ⏱ 7 min

## Der Bauplan mit Java-Resten

Falle P9 – Struktogramm nicht nach EH §5 + P6 – Off-by-one. Klassiker aus Welt 0 (Struktogramm entwickeln) und Welt 4 (dynamische Reihung). Das Struktogramm (als Text, Einrückung = Schachtelung) soll die Operation anzahlSchwach(d: dynamische Reihung vom Inhaltstyp Monster, grenze: Ganzzahl): Ganzzahl darstellen: die Anzahl der Monster mit weniger als grenze Lebenspunkten.

**a)** Nenne mindestens fünf Stellen, die nicht der Struktogramm-Notation der Ergänzenden Hinweise (§5) entsprechen, und jeweils die Regel.

**b)** Eine Stelle ist zusätzlich fachlich falsch (P6). Benenne sie und ihre Folge.

**c)** Notiere das Struktogramm korrekt und gib die Rückgabe für Monster mit den Lebenspunkten 40, 12, 90, 7 und grenze = 30 an.

```
anzahlSchwach(d, grenze)
  int anzahl = 0;
  für (i = 0; i <= d.getLength(); i++)
    m = d.getItem(i)
    falls m.getLeben() < grenze
      dann: anzahl++
      sonst: –
  return anzahl;
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-09

WARM-UP W6-10 Funktionsweise erläutern P10

Welt 6 – Endgegner · ⏱ 8 min

## Behaupten ist nicht begründen

Falle P10 – Operator verfehlt. Klassiker aus Welt 5 (Suchbaum: ausgeglichen vs. entartet, Abi 22H-B e, 23H-A d, 24N-A j) und Welt 2 (Verdopplung). Truhen mit den Nummern 50, 30, 70, 20, 40, 60, 80 werden in dieser Reihenfolge in einen leeren binären Suchbaum A eingefügt, dieselben Nummern aufsteigend sortiert (20, 30, …, 80) in einen leeren Suchbaum B.

Aufgabe aus einer Probeklausur: „Erläutern Sie das Einfügen der Nummer 45 in den Baum A. Begründen Sie, dass eine erfolgreiche Suche in A höchstens drei Vergleiche benötigt, in B dagegen bis zu sieben. Beurteilen Sie, welche Einfügereihenfolge für 1000 Truhen geeigneter ist.“

Antwort eines Schülers: „45 kommt rechts an die 40. A ist schneller, weil A ausgeglichen ist. Also ist A besser.“

**a)** Zeichne A und B.

**b)** Erläutere, warum die Antwort trotz richtiger Ergebnisse nur wenige BE erhält (Operatoren erläutern, begründen, beurteilen).

**c)** Schreibe eine vollständige Antwort mit Bezug auf die Bäume, mit Fachbegriffen (Wurzel, Ebene, Höhe, Vergleich) und mit Zahlen für 1000 Truhen.

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-10

WARM-UP W6-11 Objektkarten und Referenzen P10 Welt 6 – Endgegner · ⏱ 8 min

## Zwei Namen, ein Held

Falle P10 – Operator verfehlt. Hier: „erläutern“ mit Fachbegriffen und Bezug zum Code. Klassiker aus Welt 1 (Objektkarten nach Code, Referenzfalle, Boss 1). Held und Waffe sind die Klassen der Heldenschmiede: Held(name, maxLeben, staerke) setzt leben = maxLeben, gold = 0, waffe = null; angriffswert() = Stärke + Schaden der Waffe (0 ohne Waffe); nimmSchaden verringert leben, nie unter 0.

**a)** Zeichne die Objektkarten aller Objekte nach Zeile 10 (mit Attributwerten; Referenzen als Pfeile; inventar und anzahl darfst du weglassen) und gib an, welche Variablen auf welches Objekt verweisen.

**b)** Gib die Werte von gleich1, gleich2, a und b an.

**c)** Erläutere den Unterschied zwischen den Vergleichen in Zeile 7 und Zeile 8. Verwende die Fachbegriffe Objekt, Referenz und Identität und beziehe dich auf h, k und t.

```
1  Held h = new Held("Runa", 80, 12);
2  Waffe w = new Waffe("Drachenklinge", 120, 18);
3  Held k = h;
4  k.ausruesten(w);
5  h.nimmSchaden(30);
6  Held t = new Held("Runa", 80, 12);
7  boolean gleich1 = (h == k);
8  boolean gleich2 = (h == t);
9  int a = h.angriffswert();
10 int b = t.angriffswert();
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-11

WARM-UP W6-12 Funktionsweise erläutern P6 P10 Welt 6 – Endgegner · ⏱ 8 min

## Das Sortier-Duell der Teams

Falle P6 – Off-by-one + P10 – Operator verfehlt. Klassiker aus Welt 2 (Selectionsort, Vergleiche zählen, Verdopplung abschätzen; Abi 22N-C d, 23H-C d). Die Methode sortiert die Ruhmespunkte der Teams aufsteigend und zählt die Vergleiche zwischen Elementen der Reihung.

**a)** Gib die Belegung von r an der mit „// → Tabellenzeile“ markierten Stelle (nach jedem Durchlauf der äußeren Schleife) für r = {14, 3, 27, 9, 20} an sowie die Rückgabe.

**b)** Leite eine Formel für die Anzahl der Vergleiche bei n Elementen her und schätze ab, wie sich die Anzahl ändert, wenn n von 1000 auf 2000 verdoppelt wird.

**c)** Jemand schreibt in Zeile 5 j <= r.length. Erläutere, was passiert. Was ändert sich stattdessen, wenn Zeile 3 i < r.length lautet?

```
1  public static int sortiere(int[] r) {
2      int vergleiche = 0;
3      for (int i = 0; i < r.length - 1; i++) {
4          int min = i;
5          for (int j = i + 1; j < r.length; j++) {
6              vergleiche++;
7              if (r[j] < r[min]) {
8                  min = j;
9              }
10         }
11         int h = r[i];
12         r[i] = r[min];
13         r[min] = h;   // → Tabellenzeile
14     }
15     return vergleiche;
16 }
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-12

WARM-UP W6-13 Methode implementieren P7 P1

Welt 6 – Endgegner · ⏱ 8 min

## Die Einzelgänger im Truhenbaum

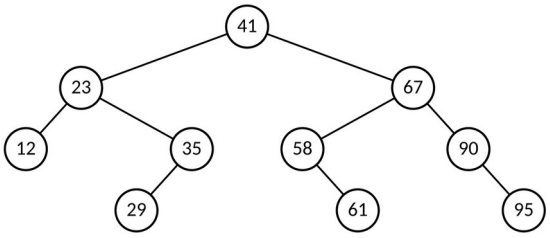
Falle P7 – Rekursion + P1 – Laufzeitfehler nicht ausgeschlossen. Klassiker aus Welt 5 (rekursive Baummethode nach dem Bauplan „leer? – links – rechts – verknüpfen“; Abi 23N-A b, 24H-A h/i). S ist ein binärer Suchbaum mit Truhennummern, entstanden durch Einfügen von 41, 23, 67, 12, 35, 58, 90, 29, 61, 95. Bibliothek „Niedersachsen“: BinTree mit isEmpty, isLeaf, getItem, getLeft, getRight; Zugriffe auf einen leeren Baum lösen einen Laufzeitfehler aus.

**a)** Implementiere einKind(b: BinTree): Ganzzahl. Die Methode liefert rekursiv die Anzahl der Knoten, die genau einen nicht leeren Teilbaum haben. Gib einKind(S) an.

**b)** Ein Mitschüler verwendet if (b.isLeaf()) { return 0; } als einzigen Basisfall und untersucht dann erst den linken, danach den rechten Teilbaum. Gib den Aufruf an, bei dem es für S zum Laufzeitfehler kommt, und erläutere die Ursache.

**c)** Eine Mitschülerin gibt die Methode unten ab. Gib ihr Ergebnis für S an und benenne den Fehler.

```
public static int einKind(BinTree b) {
    if (b.isEmpty()) {
        return 0;
    }
    int zaehler = 0;
    if (b.getLeft().isEmpty()
            != b.getRight().isEmpty()) {
        zaehler = 1;
    }
    einKind(b.getLeft());
    einKind(b.getRight());
    return zaehler;
}
```




*Baum S*

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-13

WARM-UP W6-14 Verändern und Erweitern P1 P6

Welt 6 – Endgegner · ⏱ 8 min

## Fallen im Thronsaal

Falle P1 – Laufzeitfehler nicht ausgeschlossen + P6 – Off-by-one. Klassiker aus Welt 3 (Dungeon-Karte als 2D-Reihung [Zeile][Spalte], Nachbarschaft und Randfälle; Abi 22H-C, 22N-C). Der Thronsaal ist eine Reihung von Reihung von Zeichen: '#' Wand, '^' Falle, '.' frei. fallenRingsum(karte, z, s) soll die Anzahl der Fallen auf den bis zu acht Nachbarfeldern von (z, s) liefern – das Feld (z, s) selbst zählt nicht mit.

**a)** Gib die korrekten Rückgaben für (0, 0), (1, 3), (2, 1) und (3, 5) an.

**b)** Gib für die Methode unten an, welche dieser vier Aufrufe zu einem Laufzeitfehler führen und welche ein falsches Ergebnis liefern.

**c)** Verändere die Methode so, dass sie für jedes Feld der Karte korrekt arbeitet. Nutze karte.length und karte[0].length für die Grenzen.

```
       Spalte  0 1 2 3 4 5
Zeile 0:       # . ^ . . #
Zeile 1:       . . ^ ^ . .
Zeile 2:       ^ . . . # ^
Zeile 3:       . ^ . . ^ .

public static int fallenRingsum(char[][] karte, int z, int s) {
    int anzahl = 0;
    for (int dz = -1; dz <= 1; dz++) {
        for (int ds = -1; ds <= 1; ds++) {
            if (karte[z + dz][s + ds] == '^') {
                anzahl++;
            }
        }
    }
    return anzahl;
}
```

PAPIER ohne Gerät · auf eigenem Blatt lösen · Lösung im Lösungsteil unter W6-14